Protein Design Week

PyMOL Tutorial

by Emma

PDW

What is PyMOL?

PyMOL is an Open-source software that transforms protein structures from PDB files into stunning 3D visualizations. It's your toolkit for exploring how proteins are built, understanding how atoms interact with each other, and creating publication-ready images of molecular complexes, all through an intuitive interface.

First thing first

Pymol installation video tutorial

(watch from min. 0.33 to min. 3.00)

To watch the tutorial click here

PDW

Why do we need pymol?

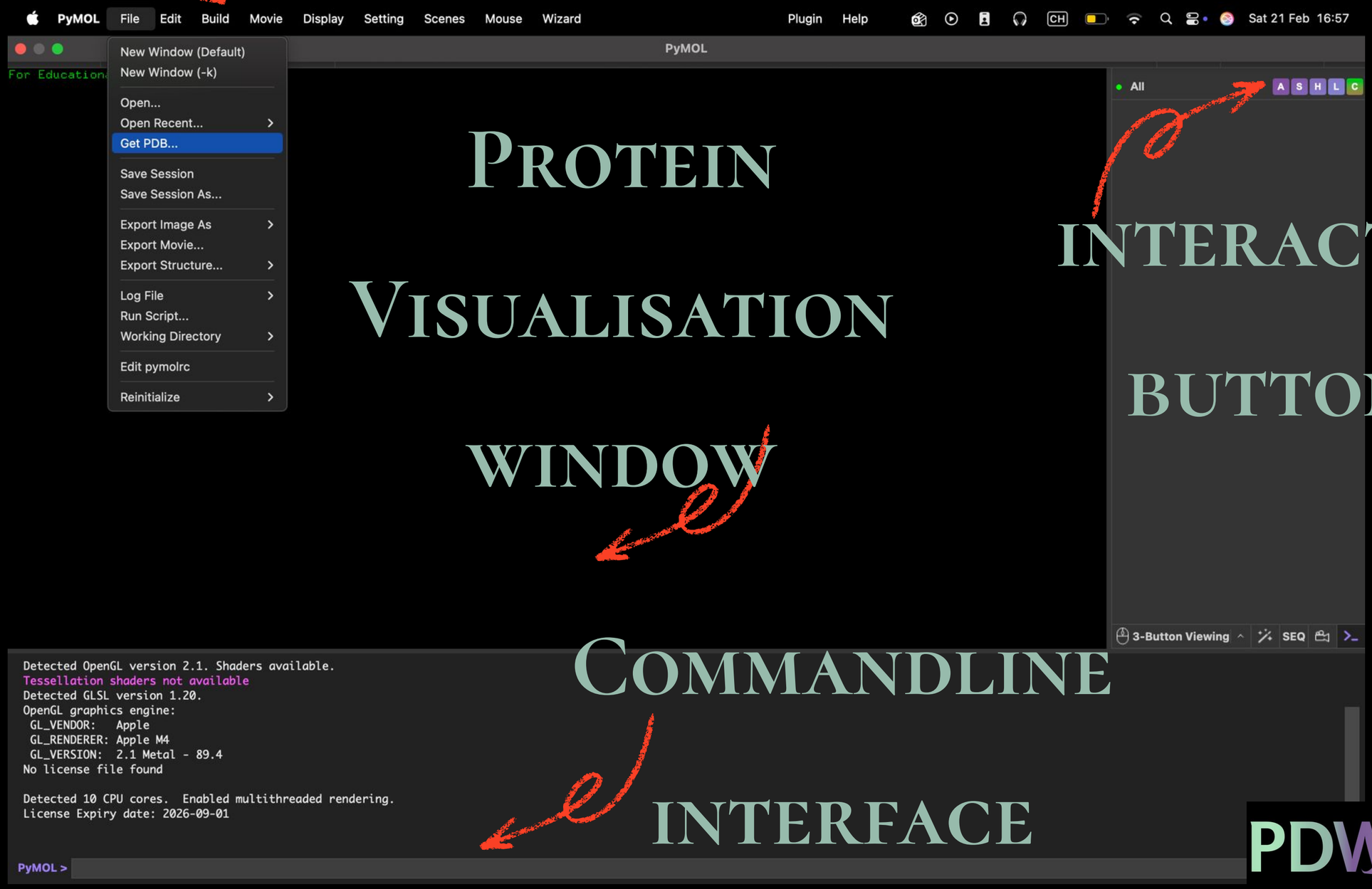
Concretely, pymol allows bioengineers to **load, visualise, manipulate and analyse 3D sturctures of proteins**. You can also do some other useful things, like merging two PDB file into one (useful for RFD3 😉)

GUI basics

PDW

GUI

Actions



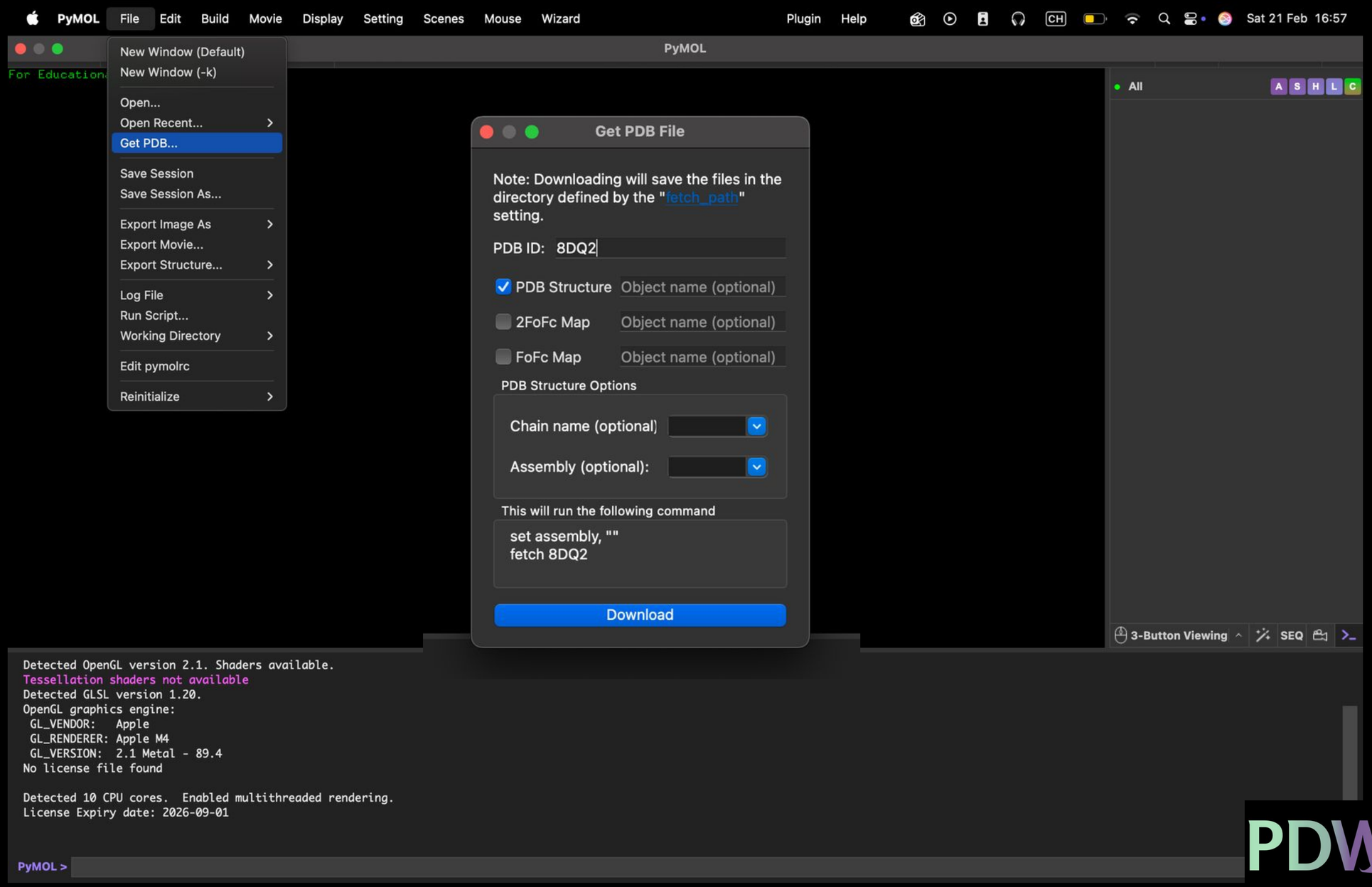
Protein Visualisation window

Interactive buttons

Commandline interface

PDW

Fetch a PDB file

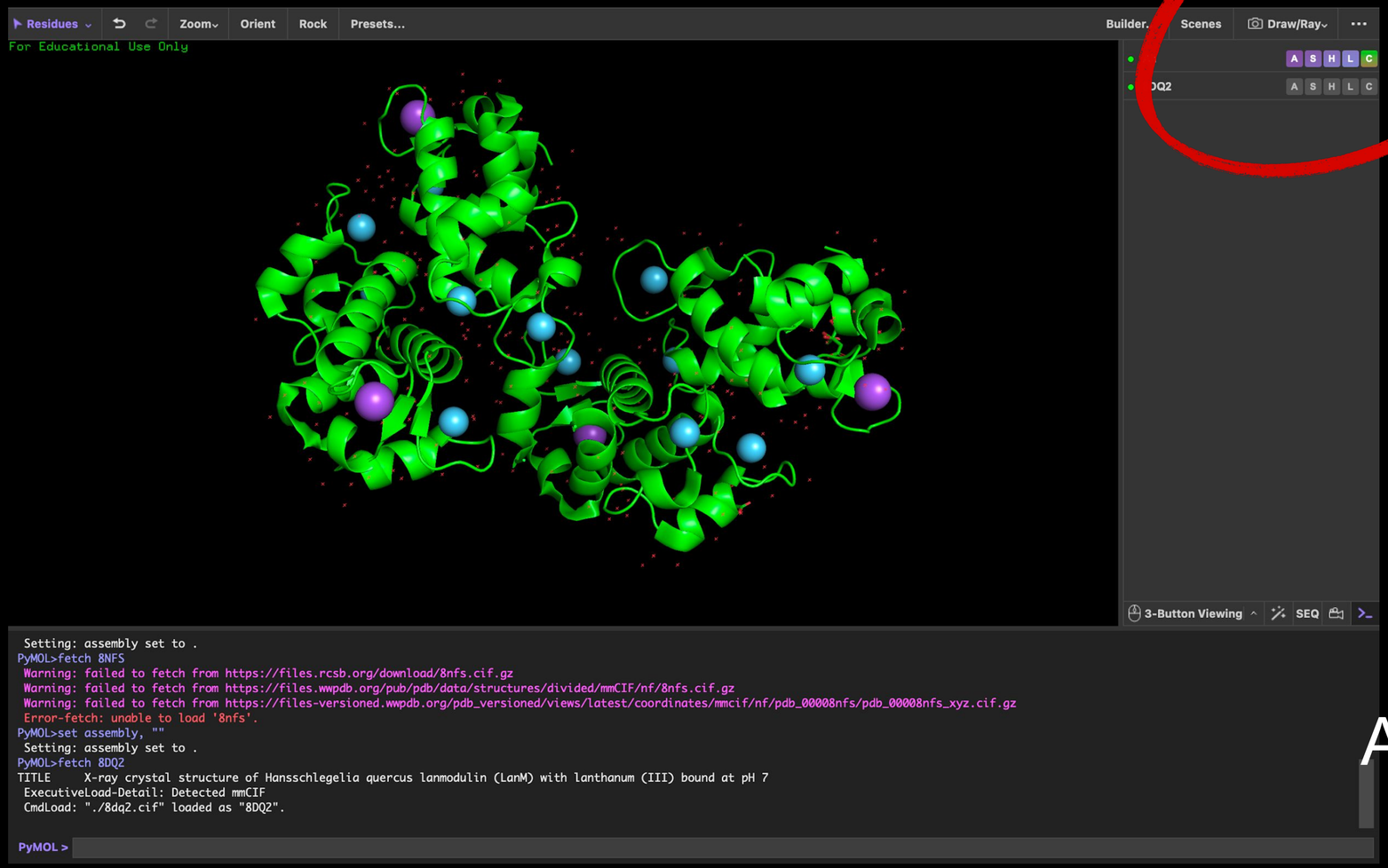


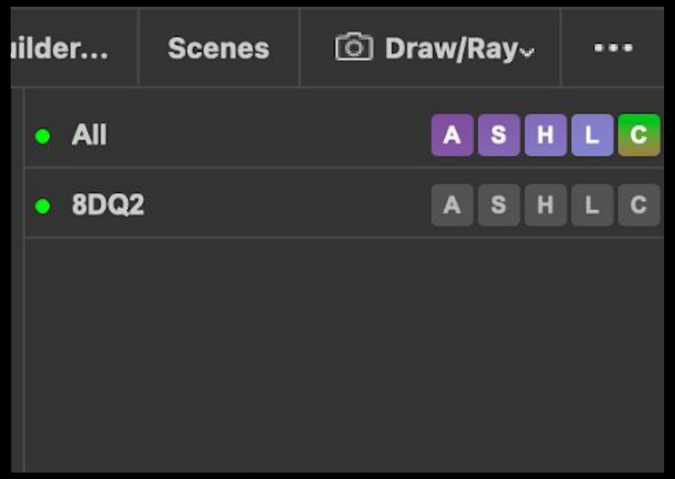
PROTEIN LOADED SUCCESSFULLY!

Your protein model

PDW

Model visualisation



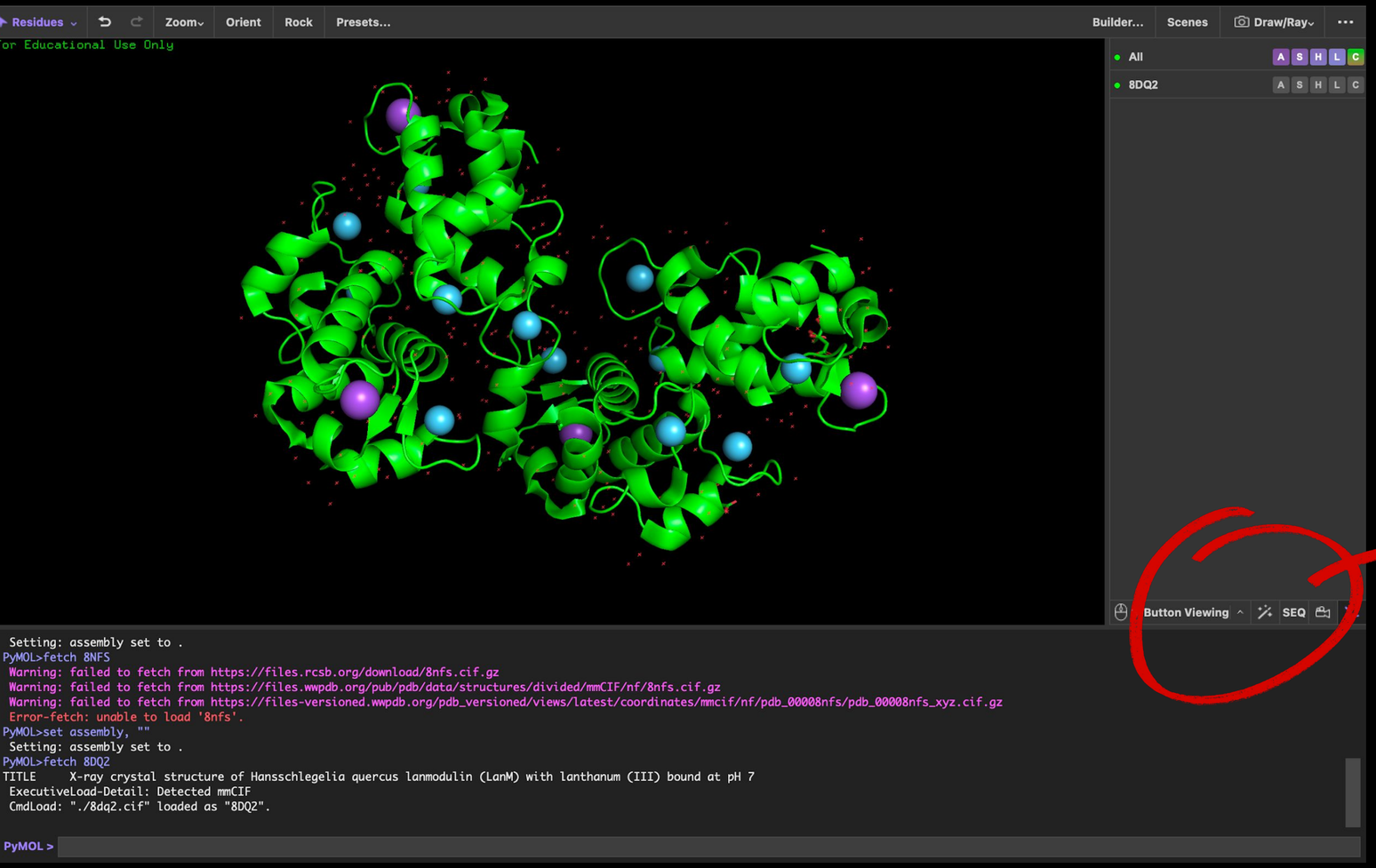


4 interactive buttons:

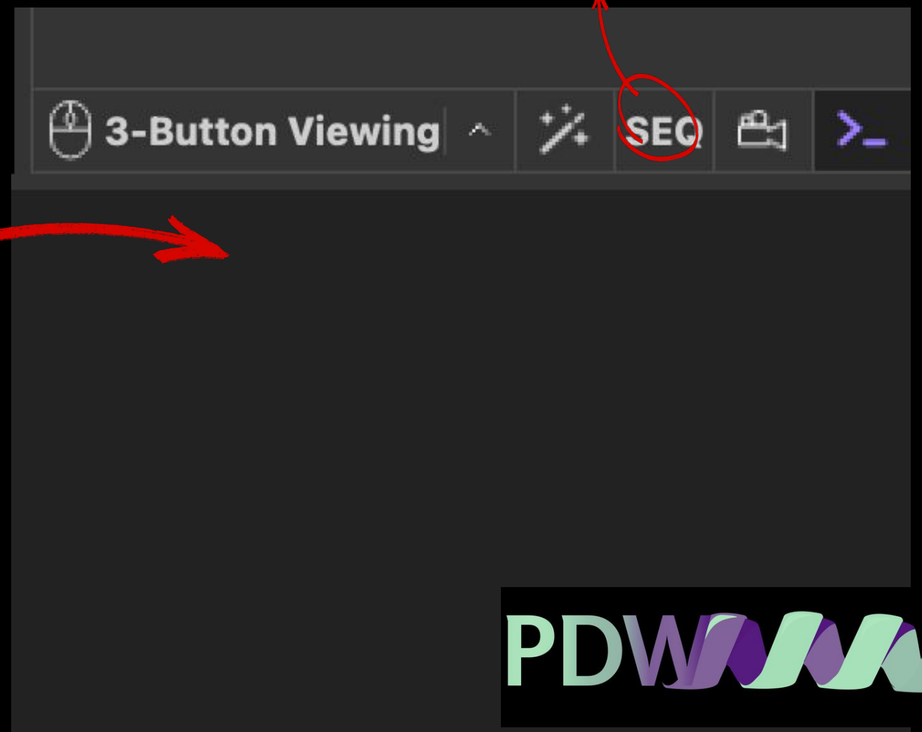
A = Action | S =show | H=Hide

| L=Label | C= Color

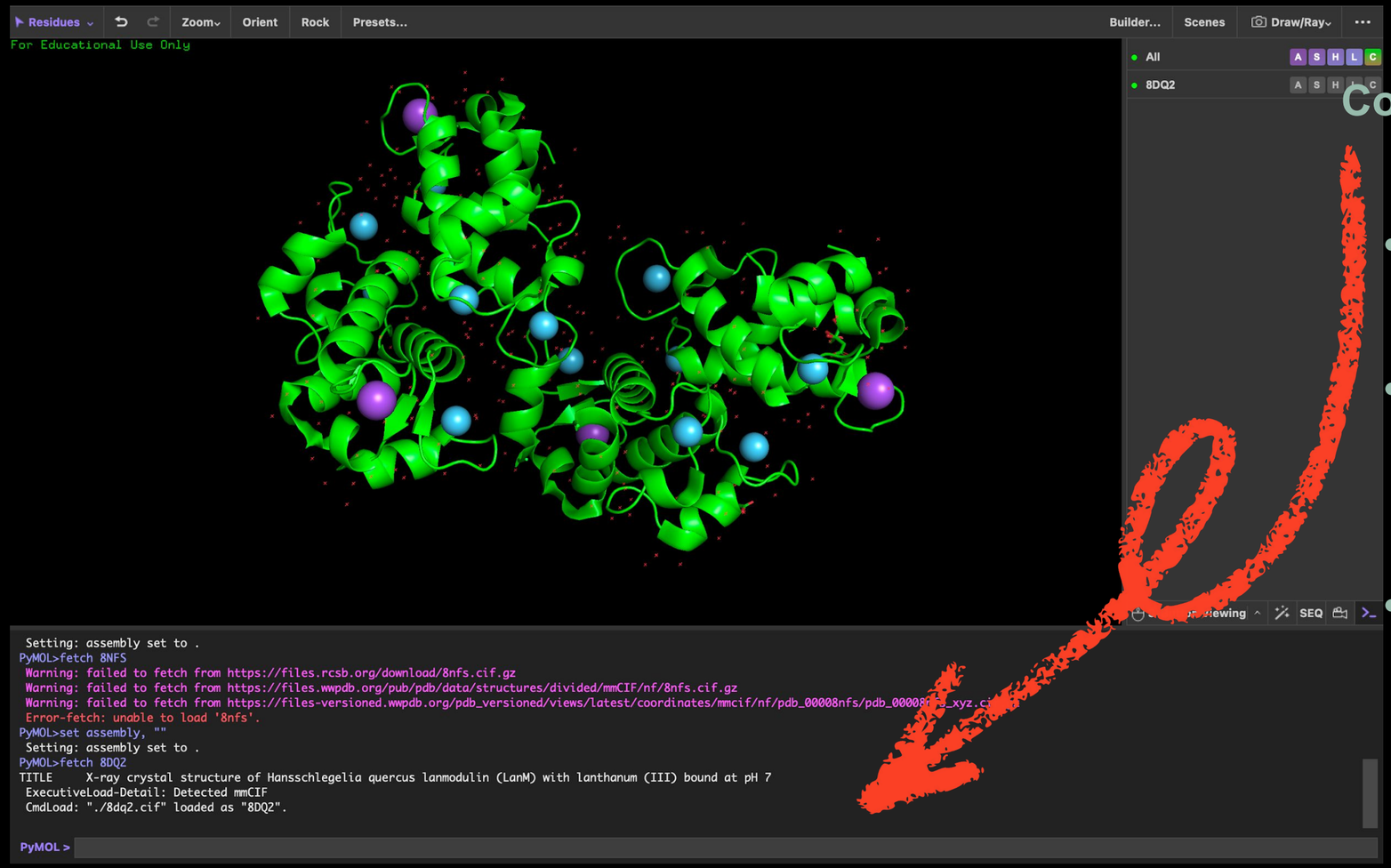
Model visualisation



Here you can select SEQ and you will see the protein's sequence



Model visualisation

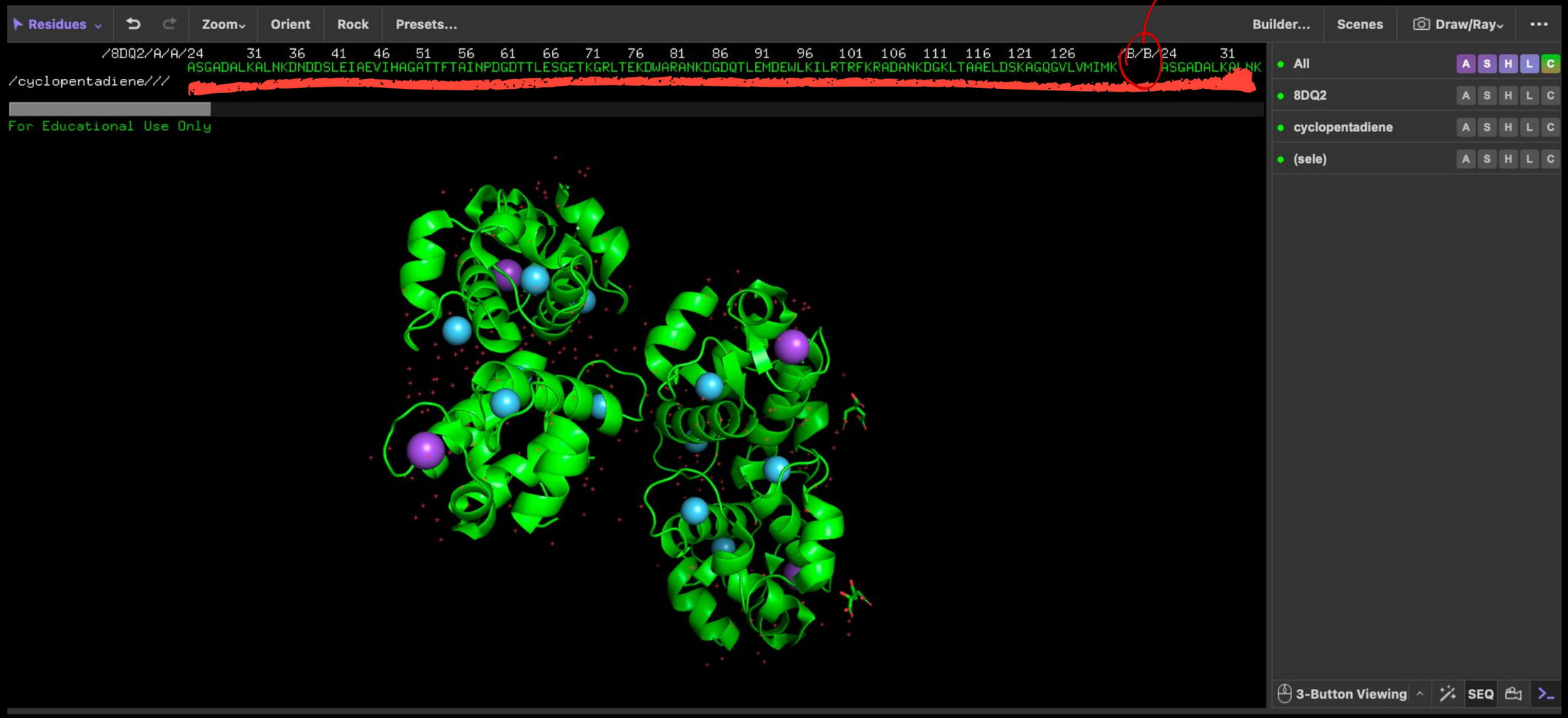


Command-Line Methods:

- **Turn on sequence viewer**: set seq_view, 1 to visualise seq (toggle off with 0)
- **View as FASTA:** print cmd.get_fastastr('all') (replace 'all' with object/selection name, e.g., 'chain A')
- **List residues in selection**: iterate your_selection (replace with your seection name), print resn (prints one-letter codes)

Model visualisation

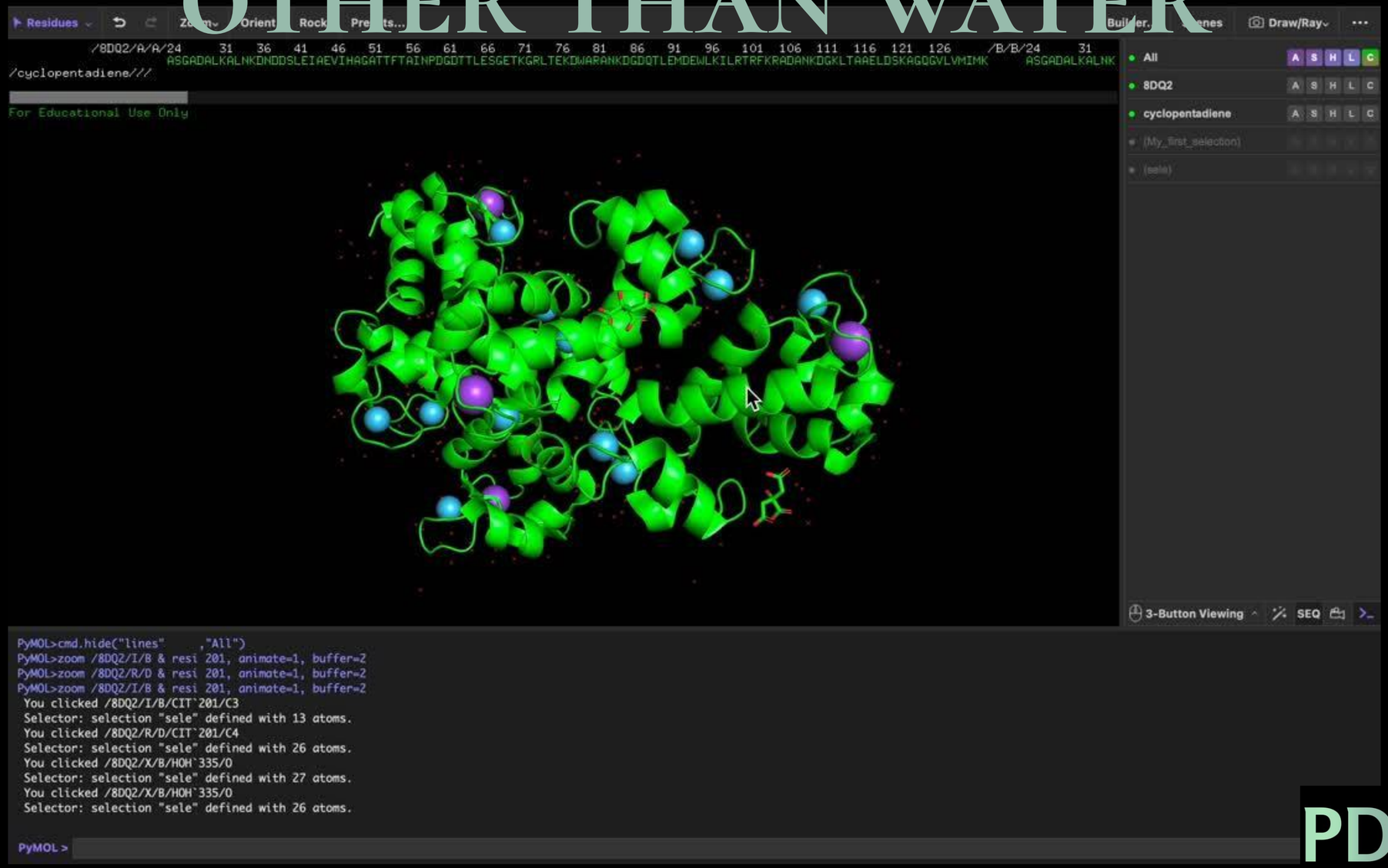
Space means chain separation



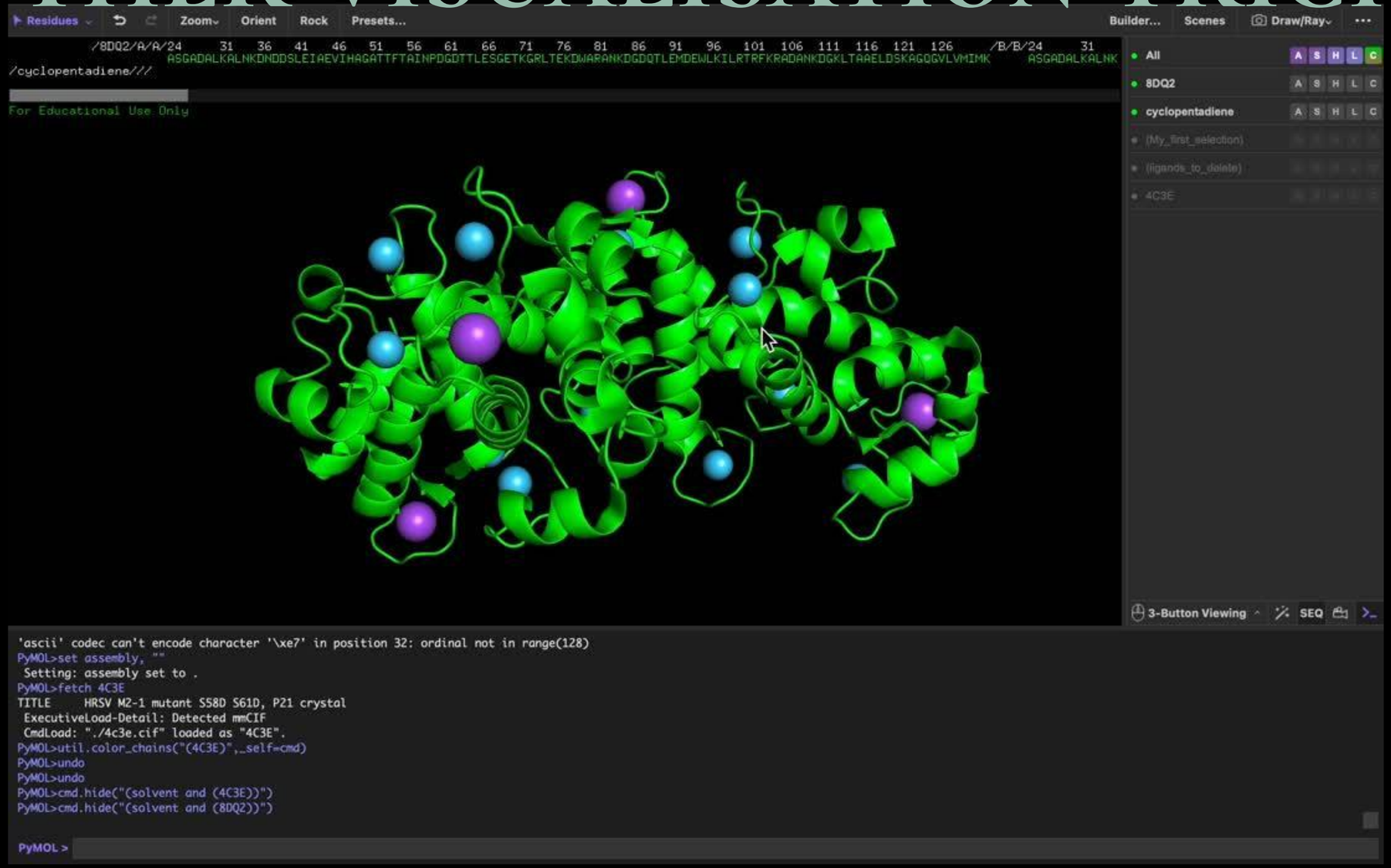
How to save a selection in a separate PDB

How to hide waters

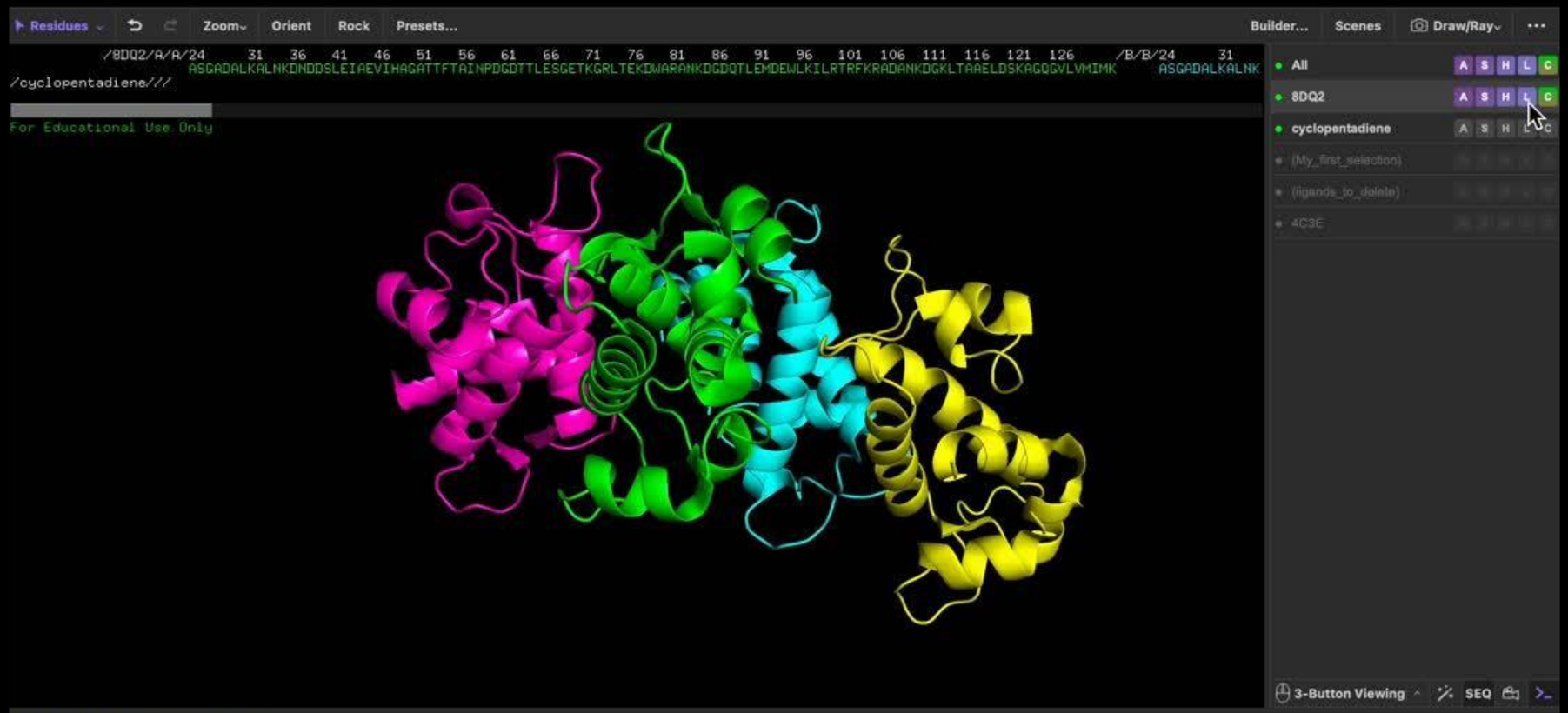
How to hide unwanted moyeties other than water



Other visualisation tricks



Labelling

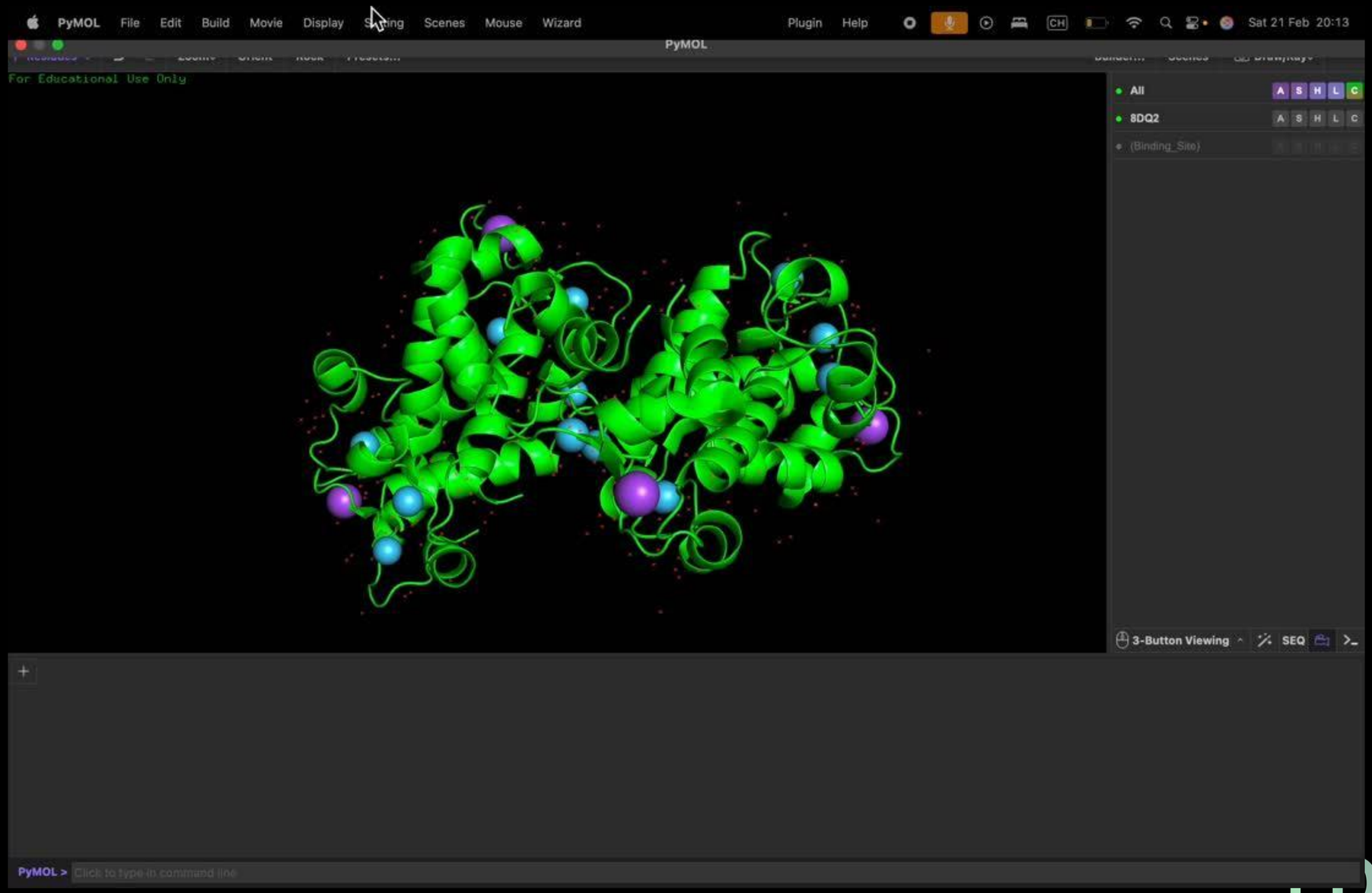


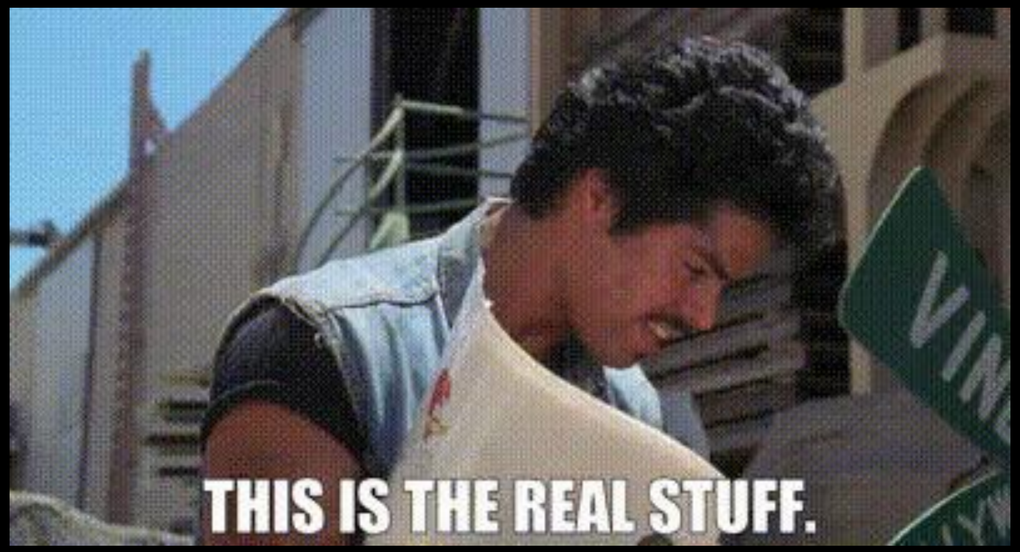

```
PyMOL>cmd.hide("everything","All")
PyMOL>cmd.show("cartoon"   ,"All")
PyMOL>cmd.show("spheres"   ,"All")
PyMOL>cmd.show("lines"     ,"All")
PyMOL>cmd.hide("spheres"   ,"All")
PyMOL>cmd.hide("cartoon"   ,"All")
PyMOL>cmd.hide("sticks"    ,"8DQ2")
PyMOL>cmd.hide("sticks"    ,"8DQ2")
PyMOL>cmd.show("cartoon"   ,"8DQ2")
PyMOL>cmd.hide("sticks"    ,"8DQ2")
PyMOL>cmd.hide("everything","8DQ2")
PyMOL>cmd.show("cartoon"   ,"8DQ2")

PyMOL >
```

You can also just label your selection by clicking L in My_firs_selection

How to export the image/video





NOW ONTO THE REAL STUFF

PDW

Rapid Structure Exploration

1 . Visualisation of some important regions (e.g.,

Binding sites, Active sites (cavities)

Command Line:

```
# Show surface (reveals pockets)
show surface, all
color white, all  # Clean background

# Highlight potential site (e.g., near ligand)
select active_site, resi 50+60+120  # Or use sequence viewer
show sticks, active_site and hetatm  # Ligand if present
color yellow, active_site
```

GUI: S → surface; select residues via sequence panel → S → sticks.

Rapid Structure Exploration

2 . Visual inspection of protein-protein interfaces

Command Line:

```
# For complexes (e.g., PDB with multiple chains)
show surface, all
color byobject, all  # Different colors per chain

# Interface residues (within 4Å)
select interface, chain A within 4 of chain B
show sticks, interface
color orange, interface
```

Reveals buried surfaces/contacts

Design Validation

1 . Visualizing proposed mutations

Command Line:

```
# Mutate residue 50 to phenylalanine
wizard mutagenesis, protein
get_wizard().set_mode("PHE")
get_wizard().do_select("A/50/")
get_wizard().apply()

# Now you have the mutation applied
show sticks, A/50/
color red, A/50/  # Highlight mutation
```

GUI :

```
1. Load protein: fetch 1ABC (or your design)
2. Wizard → Mutagenesis → Protein
3. Click on residue you want to mutate (or use sequence view)
4. Select new amino acid from dropdown list (e.g., tryptophan)
5. States panel shows unmutated (state 1) vs mutated (state 2)
6. Click through states to compare side-by-side
```

Design Validation

2 . Checking Mutation Impact on Local Contacts

Command Line:

(Visualise neighbours around your mutation)

```
# Select mutation and nearby residues
select mutant, A/50/
select neighbors, br. mutant gap 5  # All atoms within 5Å

# Show both clearly
show sticks, mutant + neighbors
color red, mutant      # Your mutation
color yellow, neighbors  # What it touches
distance hbonds, mutant, neighbors  # Show distances
```

GUI :

```
Action (A) menu → measure → distances
Then click on atoms to measure contact distances
```

What to look for:

- Red residues bumping each other → steric clash (bad)
- Yellow residues with good spacing → fine contact (good)
- H-bonds forming → stabilizing (excellent!)

Design Validation

3 . Surface Analysis for Protein-Protein Interfaces (e.g. For complexes (antibody-antigen, enzyme-substrate, etc.)

Command Line:

What you see:

- Orange sticks at contact zone → interface residues
- Surface overlap → buried area (interaction strength)
- Gaps in surface → potential binding pocket

```
# Load complex with multiple chains
fetch 1ABC  # e.g., PDB with chain A + B

# Show interface surfaces
show surface, chain A
show surface, chain B
color blue, chain A
color green, chain B

# Highlight buried interface residues (within 4Å of other chain)
select interface_A, chain A within 4 of chain B
select interface_B, chain B within 4 of chain A

show sticks, interface_A + interface_B
color orange, interface_A + interface_B  # Highlight contact zone

# Calculate interface area
distance contacts, interface_A, interface_B
```

Design Validation

4 . Identifying Clashes or Geometry Issues

Command Line:

(clash detection)

```
# Show mutation with nearby atoms
select mutant, A/50/
select nearby, br. mutant gap 4.5  # Van der Waals distance

# Visualize potential clashes
show spheres, nearby
show sticks, mutant

# Look for overlapping spheres = clash!
color red, mutant and not nearby   # Red = your mutation
color yellow, nearby               # Yellow = neighbors
```

GUI

Use sculpting tool to fix clashes.

```
1. Make your mutation (Wizard → Mutagenesis → pick residue)
2. Build → Sculpting → Activate
3. Sculpting tool relaxes geometry automatically
4. Build → Sculpting → Deactivate (done!)
```

Design Validation

4 . Identifying Clashes or Geometry Issues

Command Line:

(clash relaxation)

```
# After mutation
select mutant, A/50/
flag fix, br. mutant gap 10  # Fix backbone nearby

sculpt_activate 1ABC
sculpt_iterate all, cycles=2000  # Relax side-chains
sculpt_deactivate 1ABC

# Now save fixed structure
save relaxed.pdb, all
```

Design Validation

5 . Multiple Structure Alignment (e.g. WT vs your design)

Command Line:

Complete structural alignment

```
# Load both structures as separate objects
fetch 1ABC, wt_protein    # Wild-type (reference)
fetch 2DEF, design        # Your designed variant (mobile)

# Align design to WT
align design, wt_protein
```

Expected : Design moves to overlay WT. Console shows RMSD value (lower = more similar).

Specific residue alignment (e.g., binding sites)

```
# Align just active site or binding pocket
align design and resi 50-100, wt_protein and resi 50-100

# Backbone-only (most accurate)
align design and name CA, wt_protein and name CA

# Specific chains
align design and chain A, wt_protein and chain B
```

Design Validation

5 . 1 How to effectively visualise your structural alignment

Command Line:

```
# Color differently to compare
color blue, wt_protein
color red, design

# Show cartoon for fold comparison
show cartoon, all

# Alignment lines (see matched atoms)
align design, wt_protein, object=matches
show dashes, matches
```

Summary - PyMOL workflow

```
# Load your designed protein
fetch your_protein (or load your_file.pdb)

# Step 1: Show structure
show cartoon; color spectrum, ss

# Step 2: Pick mutation site
select design_site, resi 45+67+120
show sticks, design_site
color orange, design_site

# Step 3: Mutate
wizard mutagenesis, protein
# [Click residue → pick amino acid → apply]

# Step 4: Check contacts
select neighbors, br. sele gap 5
show sticks, neighbors
color yellow, neighbors

# Step 5: Relax if needed
sculpt_activate
sculpt_iterate all, cycles=1000
sculpt_deactivate

# Step 6: Export
save my_design.pdb, all
```

The tutorial is (finally) over!

Now it’s time for you to go make some cool design and show them all :)

Hopefully now you learned something!

If you still have any questions, coaches are around ;)

BUT! Before asking one of us, go through the usual drill:

1. Google it
2. Ask to your favourite AI agent
3. If you found nothing, then ask to the (not-so) all-knowing coaches